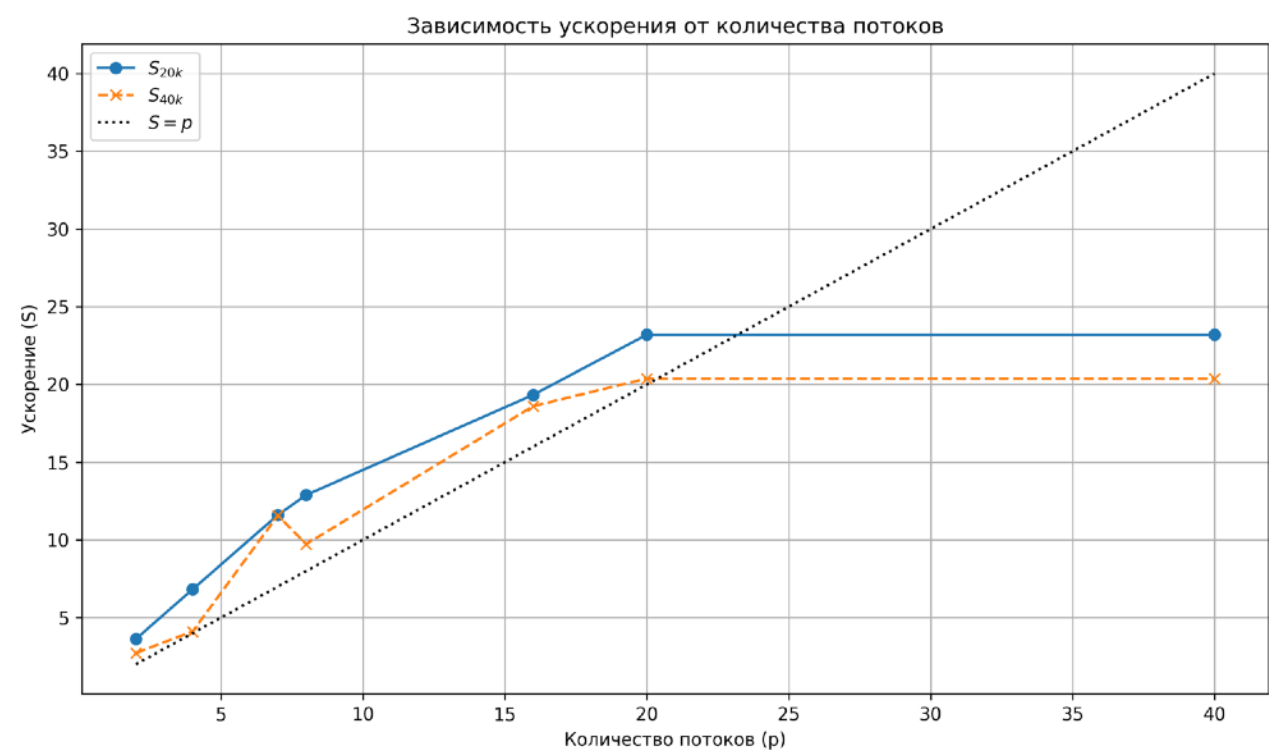


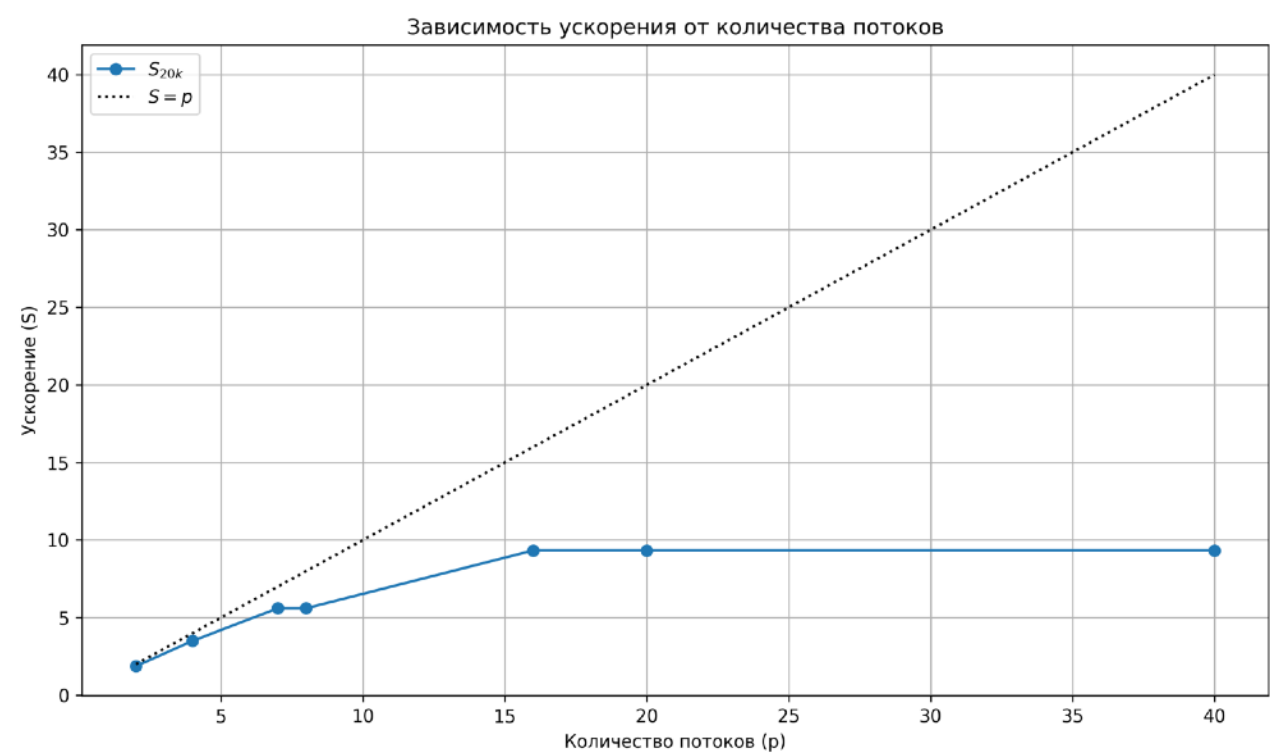
Задание №1



M=N	Количество потоков														
	2			4		7		8		16		20		40	
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40
20000 (~3 GiB)	1,16	0,32	3,62	0,17	6,82	0,1	11,6	0,09	12,89	0,06	19,33	0,05	23,2	0,05	23,2
40000 (~ 12GiB)	4,28	1,57	2,73	1,04	4,12	0,37	11,57	0,44	9,73	0,23	18,61	0,21	20,38	0,21	20,38

Вывод: падение масштабируемости после числа потоков, превышающего 20

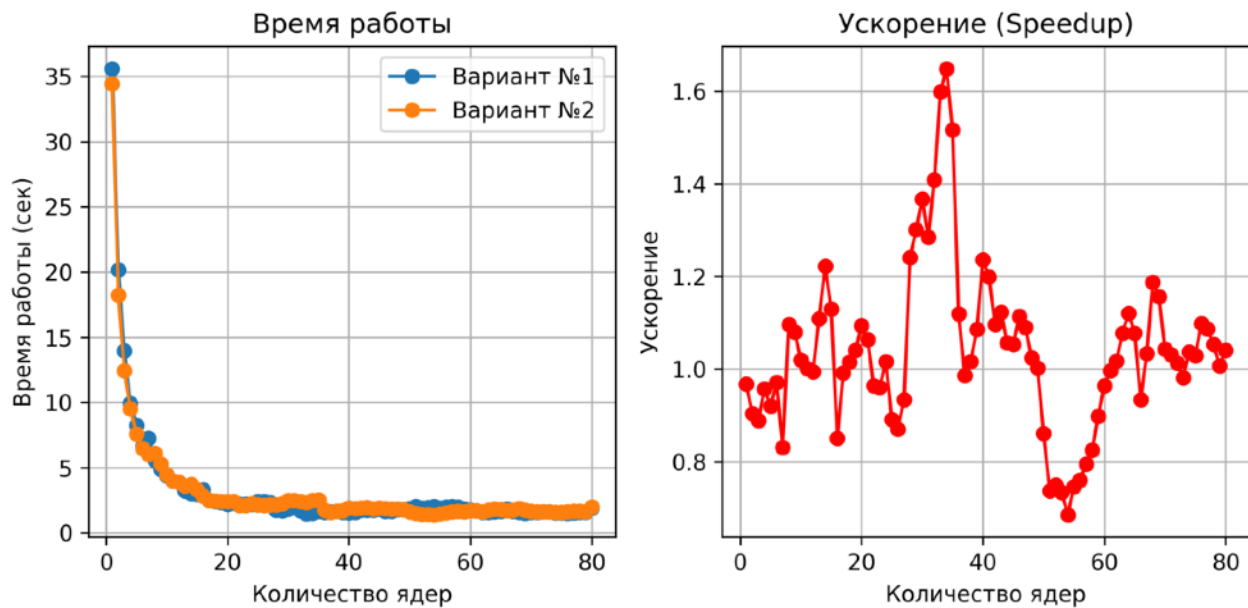
Задание №2



M=N	Количество потоков														
	2			4		7		8		16		20		40	
	T1	T2	S2	T4	S4	T7	S7	T8	S8	T16	S16	T20	S20	T40	S40
40000 (~ 12 GiB)	0,28	0,15	1,87	0,08	3,5	0,05	5,6	0,05	5,6	0,03	9,33	0,03	9,33	0,03	9,33

Вывод: падение масштабируемости после числа потоков, превышающего 7

Задание №3



Данные при тестировании:

Tau = 0.00016
Epsilon = 1e-5
N=M = 20000

Speedup[i] = $T1_i / T2_i$, где T1 - массив с временем выполнения варианта №1
T2 - массив с временем выполнения варианта №2

Вывод: 62,5% тестов быстрее при варианте №2, следовательно, целесообразнее использовать вариант №2.

#pragma omp for schedule(...):

Данные при тестировании:

N = 200 — — — — — высота / ширина матрицы
num_threads = 80 — количество ядер

Результат:

Schedule: static, Chunk: 1, Итераций: 133, Время: 3.24639 сек
Schedule: static, Chunk: 10, Итераций: 127, Время: 3.04123 сек
Schedule: static, Chunk: 100, Итераций: 148, Время: 2.95834 сек
Schedule: static, Chunk: 1000, Итераций: 147, Время: 4.16111 сек
Schedule: dynamic, Chunk: 1, Итераций: 138, Время: 5.1649 сек
Schedule: dynamic, Chunk: 10, Итераций: 162, Время: 4.72673 сек
Schedule: dynamic, Chunk: 100, Итераций: 142, Время: 4.61328 сек
Schedule: dynamic, Chunk: 1000, Итераций: 145, Время: 5.49855 сек
Schedule: guided, Chunk: 1, Итераций: 138, Время: 3.92277 сек
Schedule: guided, Chunk: 10, Итераций: 156, Время: 4.50247 сек
Schedule: guided, Chunk: 100, Итераций: 167, Время: 4.89624 сек
Schedule: guided, Chunk: 1000, Итераций: 141, Время: 5.33514 сек

Минимальное время оказалось при использовании static при использовании 100 чанков (2.95834 сек).

$A * x = b$

Возможно это произошло из-за того, что static предназначен для алгоритмов, в которых нагрузка между итерациями распределена равномерно (как в случае текущей задачи: матрица **A** с доминирующей диагональю: 2, остальные элементы: 1, результирующий массив **b** однороден и каждый элемент в нем равен N+1, начальное значение вектора **x** который мы ищем состоит из 0)

Характеристики системы:

1. Lscpu:

Model name: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
CPU family: 6
Model: 85
Thread(s) per core: 2
Core(s) per socket: 20
Socket(s): 2
Stepping: 7
CPU max MHz: 3900.0000
CPU min MHz: 1000.0000

2. Наименование сервера:

ProLiant XL270d Gen10

3. Сколько NUMA:

available: 2 nodes (0-1)

4. сколько памяти у каждой ноды:

node 0 size: 385636 MB
node 0 free: 269440 MB
node 1 size: 387008 MB
node 1 free: 190932 MB

5. операционная система

Ubuntu 22.04.5 LTS